Ethereum Fraud Detection

using Machine Learning

Technical Documentation Report

Capstone Project • AI-Powered Blockchain Security

By:

Kareem Ashraf Talaat Sayed

Ahmed Ahmed Abdel-Salam Ahmed
Ali ElDin Tarek Ali Ibrahim
Ahmed Fouad Mady Mohamed
Mostafa Mahmoud Ahmed Mohamed

Faculty Of Engineering
[Capital University]

Academic Year 2026

Abstract

Fraudulent activity on the Ethereum blockchain represents a significant and growing threat to the integrity of decentralised finance. This report presents a comprehensive machine learning system designed to classify Ethereum wallet addresses as either fraudulent or legitimate based on on-chain transactional behaviour. The system leverages a curated dataset of 9,841 labelled Ethereum account records, encompassing 47 engineered behavioural and statistical features derived from native ETH transfers and ERC-20 token movements.

Six distinct machine learning architectures were trained, evaluated, and compared: XGBoost, LightGBM, Random Forest, Logistic Regression, K-Nearest Neighbours (KNN), and a TensorFlow Deep Neural Network (DNN). A Safety Ensemble was additionally constructed by aggregating model votes with a majority-rule scheme tuned for high fraud sensitivity. The best-performing individual model, XGBoost, achieved 99.49% accuracy with a fraud recall of 97.71%. The Safety Ensemble achieved 99.58% accuracy and 98.50% fraud recall, making it the most suitable configuration for production deployment.

The system is deployed as an interactive Streamlit web application that enables real-time per-transaction fraud scoring and multi-model comparison. The report documents the theoretical foundations, implementation details, evaluation methodology, and deployment architecture of the complete system.

1. Introduction

1.1 Ethereum and the Blockchain Ecosystem

Ethereum is an open-source, decentralised blockchain platform launched in 2015 by Vitalik Buterin. Unlike Bitcoin, which was designed primarily as a peer-to-peer electronic cash system, Ethereum introduced a Turing-complete virtual machine — the Ethereum Virtual Machine (EVM) — enabling developers to deploy self-executing programmes called smart contracts. These contracts underpin the broader ecosystem of decentralised applications (dApps), decentralised finance (DeFi), and non-fungible tokens (NFTs). As of 2024, Ethereum hosts over $40 billion in total value locked (TVL) across its DeFi protocols and processes millions of transactions daily.

Every Ethereum transaction is permanently recorded on a public ledger, creating a transparent and immutable audit trail. Accounts (wallet addresses) interact by sending Ether (ETH) — the native currency — and ERC-20 tokens, which are fungible tokens conforming to a standardised smart contract interface. This rich, publicly accessible transaction history forms the foundation of the feature engineering performed in this project.

1.2 The Problem of Blockchain Fraud

Despite its transparency, the pseudonymous nature of Ethereum addresses creates significant opportunities for illicit activity. Common forms of Ethereum-based fraud include Ponzi schemes, rug pulls, phishing scams, money laundering, and exchange hacks. In a rug pull, developers of a token project artificially inflate token value before withdrawing all liquidity and abandoning investors. Phishing attackers deploy counterfeit contract addresses or front-end interfaces to steal funds. According to Chainalysis, illicit cryptocurrency addresses received over $24 billion in 2023 alone.

Traditional fraud detection methods rely on centralised Know-Your-Customer (KYC) checks and rule-based transaction monitoring. These approaches are fundamentally inadequate in a permissionless, pseudonymous blockchain environment where anyone can generate a new address instantly and where transaction volumes make manual review infeasible.

1.3 Role of Machine Learning

Machine learning offers a powerful alternative: rather than relying on predefined rules, ML models learn discriminative patterns from historical labelled data. On-chain transaction behaviour of fraudulent addresses often exhibits statistically anomalous patterns — unusually

rapid fund movement, abnormal transaction frequency distributions, or interaction with a concentrated set of counterpart addresses. These patterns manifest as statistical features computable directly from the public Ethereum ledger, enabling supervised binary classification.

This project applies six diverse ML architectures — spanning linear models, ensemble tree methods, instance-based learning, and deep learning — to this binary classification task, thereby providing both a practical fraud detection solution and a comparative study of ML methodologies on blockchain data.

2. Problem Definition

The core task is a supervised binary classification problem formally defined as follows:

Given a feature vector $x \in \mathbb{R}^d$ representing the aggregated transaction statistics of an Ethereum wallet address, predict the binary label $y \in \{0, 1\}$, where:

- $y = 0$ indicates a legitimate (non-fraudulent) address
- $y = 1$ indicates a fraudulent (illicit) address

The model is trained on a labelled dataset $D = \{(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)\}$ where $n = 9{,}841$ and the feature dimension $d = 47$ (after preprocessing). The task objective is to learn a mapping $f : \mathbb{R}^d \rightarrow \{0, 1\}$ that minimises misclassification, with particular emphasis on minimising false negatives (missed fraud cases), as failing to detect a fraudulent address carries substantially higher real-world cost than a false alarm.

This asymmetric cost structure motivates the use of the Safety Ensemble with a majority-vote threshold of ≥ 2 positive votes out of 6 models, increasing sensitivity at the cost of a controlled increase in false positive rate.

3. Dataset Description

3.1 Overview

The dataset used in this project is a curated collection of Ethereum account transaction records sourced from the Ethereum blockchain via Etherscan's public API. The dataset contains 9,841 labelled addresses and 51 raw columns (including administrative fields), yielding 47 usable feature columns after removing identifiers and the target label.

Property	Value
Total Records	9,841
Legitimate Accounts (FLAG = 0)	7,662 (77.9%)
Fraudulent Accounts (FLAG = 1)	2,179 (22.1%)
Raw Feature Columns	47 (after dropping Address, Index, FLAG)
Categorical Features	2 (ERC-20 most sent token type, most received token type)
Numeric Features	45
Total Missing Values	22,635 (filled with 0)
Target Variable	FLAG (0 = Legitimate, 1 = Fraud)

3.2 Feature Descriptions

The 47 features fall into three major categories:

3.2.1 Native ETH Transaction Features

These features summarise the behaviour of the account with respect to standard ETH transfers:

Feature	Description
Avg min between sent tnx	Average time gap (minutes) between consecutive outgoing transactions
Avg min between received tnx	Average time gap (minutes) between consecutive incoming transactions
Time Diff between first and last (Mins)	Total active lifespan of the account in minutes
Sent tnx	Total number of outgoing transactions
Received Tnx	Total number of incoming transactions

Feature	Description
Number of Created Contracts	Number of smart contracts deployed by this address
Unique Received From Addresses	Number of distinct counterpart addresses that sent ETH to this account
Unique Sent To Addresses	Number of distinct counterpart addresses this account sent ETH to
min / max / avg val received	Statistical summary of ETH received per transaction
min / max / avg val sent	Statistical summary of ETH sent per transaction
min / max / avg value sent to contract	Statistics of ETH sent specifically to contract addresses
total transactions	Total transaction count including contract creation transactions
total Ether sent / received	Aggregate ETH volume transferred out / in
total ether sent contracts	Total ETH value sent to smart contracts
total ether balance	Current net ETH balance of the account

3.2.2 ERC-20 Token Transaction Features

ERC-20 features capture behaviour in the token economy, which is where many sophisticated fraud schemes operate:

Feature	Description
Total ERC20 tnxs	Total number of ERC-20 token transfer events
ERC20 total Ether received / sent	Total ETH-equivalent value of ERC-20 tokens received / sent
ERC20 total Ether sent contract	ETH-equivalent ERC-20 tokens sent to contract addresses
ERC20 uniq sent addr	Number of unique addresses tokens were sent to
ERC20 uniq rec addr	Number of unique addresses tokens were received from
ERC20 uniq rec contract addr	Number of unique contract addresses tokens were received from
ERC20 avg time between sent / rec tnx	Average time between ERC-20 send / receive events
ERC20 min / max / avg val rec / sent	Statistical summaries of ERC-20 token amounts per transaction
ERC20 uniq sent / rec token name	Number of distinct token types sent / received
ERC20 most sent token type	Name of ERC-20 token most frequently sent (categorical)

Feature	Description
ERC20_most_rec_token_type	Name of ERC-20 token most frequently received (categorical)

3.3 Class Imbalance

The dataset exhibits a moderate class imbalance: approximately 78% legitimate addresses versus 22% fraudulent addresses. While this ratio is not extreme, it motivates the use of metrics beyond raw accuracy — specifically, recall on the fraud class — and the design of the Safety Ensemble with a low fraud-vote threshold to maximise sensitivity.

3.4 Preprocessing Pipeline

The preprocessing steps applied in the system are as follows:

- Column name normalisation: spaces replaced with underscores to prevent pandas parsing errors.
- Dropping administrative columns: Unnamed: 0, Index, and Address are removed as they carry no predictive signal.
- Missing value imputation: All NaN entries are filled with 0, treating missing ERC-20 activity as absent.
- Categorical encoding: The two text columns (ERC20 most sent and received token type) are converted to binary indicator columns via pandas get_dummies(), expanding the feature space.
- Feature alignment: After one-hot encoding, any feature columns present in the training scaler but absent in the inference input are added with value 0 to ensure dimensional consistency.
- Standard scaling: All features are transformed using a fitted StandardScaler, centring each feature to zero mean and unit variance. The scaler is serialised to scaler.pkl and reused at inference time.

4. System Pipeline and Architecture

4.1 End-to-End Workflow

The complete machine learning pipeline follows a structured sequence of stages from raw data acquisition to real-time prediction. Each stage is described below:

Stage 1: Data Collection

Ethereum transaction records are collected from Etherscan's public API. Each record corresponds to a unique wallet address and aggregates all transaction activity associated with that address into summary statistics. Ground-truth fraud labels (FLAG) are obtained from known blacklists and community-reported fraudulent addresses.

Stage 2: Exploratory Data Analysis

Prior to modelling, the dataset is explored to understand feature distributions, identify missing values, assess class balance, and detect potential data quality issues. Visualisations including class distribution pie charts and feature correlation matrices inform preprocessing decisions.

Stage 3: Data Preprocessing

As detailed in Section 3.4, preprocessing involves column normalisation, identifier removal, missing value imputation, categorical encoding, feature alignment, and standard scaling. The fitted scaler is saved to disk to ensure identical transformations at inference time.

Stage 4: Model Training

Six models are trained independently on the preprocessed and scaled dataset. Each model is trained using an 80/20 stratified train-test split to preserve the class ratio across partitions. Hyperparameters are selected via a combination of domain knowledge and cross-validation.

Stage 5: Model Evaluation

Each trained model is evaluated on the held-out test set using accuracy, precision, recall, F1-score, and confusion matrix analysis. The fraud recall metric is given primary importance given the asymmetric cost of false negatives.

Stage 6: Model Serialisation

All trained sklearn-compatible models (XGBoost, LightGBM, Random Forest, Logistic Regression, KNN) are serialised using joblib.dump() and saved as .pkl files. The TensorFlow

DNN is saved in the .keras format using model.save(). The fitted StandardScaler is also serialised.

Stage 7: Deployment (Streamlit Application)

The trained models and scaler are loaded into a Streamlit web application at startup via cached resource functions. The application provides three tabs: a dataset browser, a performance metrics dashboard, and a live detection interface that samples random transactions and displays multi-model predictions alongside the Safety Ensemble verdict.

5. Machine Learning Models

This section provides an in-depth treatment of each model architecture employed in the project, covering theoretical foundations, implementation details, and results.

5.1 XGBoost — eXtreme Gradient Boosting

5.1.1 Conceptual Explanation

XGBoost (eXtreme Gradient Boosting) is a scalable, regularised implementation of gradient boosted decision trees. Gradient boosting constructs an ensemble of weak learners — typically shallow decision trees — sequentially, where each successive tree corrects the residual errors of the ensemble built so far.

Formally, the model predicts $\hat{y}_i = \Sigma_k f_k(x_i)$, where f_k is the k-th regression tree. The objective function minimised during training is:

$$Obj(\theta) = \Sigma_i\, l(y_i, \hat{y}_i) + \Sigma_k\, \Omega(f_k)$$

where l is the loss function (binary cross-entropy for classification), and $\Omega(f) = \gamma T + \frac{1}{2}\lambda||w||^2$ is a regularisation term penalising tree complexity (T = number of leaves, w = leaf weights). XGBoost approximates the optimal tree at each step using a second-order Taylor expansion of the loss, enabling efficient closed-form leaf weight computation.

Key strengths include built-in L1/L2 regularisation to prevent overfitting, handling of sparse inputs and missing values natively, column subsampling for variance reduction, and highly optimised parallelised computation via a histogram-based split-finding algorithm.

5.1.2 Suitability for Fraud Detection

XGBoost is particularly well-suited to tabular fraud detection tasks because: (1) it automatically discovers complex non-linear interactions between behavioural features without manual feature

engineering; (2) its regularisation prevents overfitting on the noisy, imbalanced fraud dataset; (3) it provides native feature importance scores enabling model interpretability; and (4) it has demonstrated state-of-the-art performance on tabular classification benchmarks with imbalanced classes.

5.1.3 Implementation

```
from xgboost import XGBClassifier
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import joblib
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42, stratify=y)
```

```
xgb_model = XGBClassifier(
    n_estimators=200,
    max_depth=6,
    learning_rate=0.1,
    subsample=0.8,
    colsample_bytree=0.8,
    use_label_encoder=False,
    eval_metric='logloss',
    random_state=42
)
xgb_model.fit(X_train, y_train)
joblib.dump(xgb_model, 'xgboost_model.pkl')
```

5.1.4 Results

Metric	Value
Accuracy	99.49%
Fraud Recall	97.71%
Model Complexity	High (ensemble of 200 trees)

5.2 LightGBM — Light Gradient Boosting Machine

5.2.1 Conceptual Explanation

LightGBM is a gradient boosting framework developed by Microsoft that introduces two key algorithmic innovations over standard GBDT: Gradient-based One-Side Sampling (GOSS) and Exclusive Feature Bundling (EFB). GOSS retains data instances with large gradients (high training error) in full and randomly samples those with small gradients, reducing the effective

dataset size while preserving statistical accuracy. EFB bundles mutually exclusive sparse features into dense ones, dramatically reducing the feature dimensionality for split finding.

Unlike XGBoost's level-wise (breadth-first) tree growth, LightGBM grows trees leaf-wise (best-first), selecting the leaf with maximum loss reduction at each step. This produces deeper, more asymmetric trees that converge faster but may overfit without depth constraints.

5.2.2 Suitability for Fraud Detection

LightGBM's leaf-wise growth strategy often captures subtle fraud patterns more efficiently than level-wise alternatives. Its handling of sparse ERC-20 token features (where most addresses have zero activity) is particularly beneficial given the structure of this dataset. Training speed is also advantageous when iterating on hyperparameter configurations.

5.2.3 Implementation

```
from lightgbm import LGBMClassifier
```

```
lgbm_model = LGBMClassifier(
    n_estimators=200,
    learning_rate=0.05,
    num_leaves=63,
    max_depth=-1,
    subsample=0.8,
    colsample_bytree=0.8,
    random_state=42
)
lgbm_model.fit(X_train, y_train)
joblib.dump(lgbm_model, 'lightgbm_model.pkl')
```

5.2.4 Results

Metric	Value
Accuracy	99.44%
Fraud Recall	97.71%
Model Complexity	High (leaf-wise ensemble)

5.3 Random Forest

5.3.1 Conceptual Explanation

Random Forest is a bagging ensemble of decision trees introduced by Leo Breiman (2001). During training, B trees are grown independently on bootstrap samples of the training data. At

each split node, only a random subset of m features (typically $m = \sqrt{d}$ for classification) is considered as split candidates, injecting additional variance between trees. Final predictions are made by majority vote across all B trees.

The variance reduction achieved by averaging over many decorrelated trees is described by:

$$Var(\bar{y}) = \rho\sigma^2 + (1-\rho)\sigma^2/B$$

where ρ is the pairwise correlation between trees and σ^2 is the variance of a single tree. Feature subsampling reduces ρ, making the ensemble variance approach $(1-\rho)\sigma^2/B \approx 0$ as $B \to \infty$.

5.3.2 Suitability for Fraud Detection

Random Forest is robust to overfitting through its averaging mechanism, handles missing values and mixed feature types well, and provides reliable out-of-bag (OOB) error estimates without a separate validation split. It is also naturally resistant to noisy features, as any single irrelevant feature is unlikely to be selected at every split across all trees.

5.3.3 Implementation

```
from sklearn.ensemble import RandomForestClassifier
```

```
rf_model = RandomForestClassifier(
    n_estimators=200,
    max_depth=None,
    min_samples_split=2,
    min_samples_leaf=1,
    max_features='sqrt',
    random_state=42,
    n_jobs=-1
)
rf_model.fit(X_train, y_train)
joblib.dump(rf_model, 'random_forest_model.pkl')
```

5.3.4 Results

Metric	Value
Accuracy	99.09%
Fraud Recall	95.87%
Model Complexity	Medium-High (200 trees, full depth)

5.4 Logistic Regression

5.4.1 Conceptual Explanation

Logistic Regression is a linear discriminative classifier that models the posterior probability of class membership using the sigmoid (logistic) function:

```
P(y=1 | x) = σ(wᵀx + b) = 1 / (1 + exp(-(wᵀx + b)))
```

The model parameters $w \in \mathbb{R}^d$ and $b \in \mathbb{R}$ are estimated by maximising the log-likelihood of the training labels, equivalent to minimising the binary cross-entropy loss with an optional L2 regularisation penalty:

```
L(w, b) = -Σᵢ [yᵢ log(ŷᵢ) + (1-yᵢ) log(1-ŷᵢ)] + (λ/2)||w||²
```

The decision boundary is a hyperplane $w^T x + b = 0$ in the feature space. Despite its simplicity, Logistic Regression is a strong baseline and is highly interpretable: each weight w_j represents the log-odds change in the fraud probability per unit increase in feature j.

5.4.2 Suitability for Fraud Detection

Logistic Regression serves as an important baseline, providing an upper bound on what a linear model can achieve. It is computationally fast, interpretable, and well-calibrated in terms of probability outputs. Its performance on this dataset (97.77% accuracy) quantifies the linearly separable component of the fraud signal, while the gap to ensemble methods indicates the value added by non-linear feature interaction modelling.

5.4.3 Implementation

```
from sklearn.linear_model import LogisticRegression
```

```
lr_model = LogisticRegression(
    C=1.0,
    solver='lbfgs',
    max_iter=1000,
    random_state=42
)
lr_model.fit(X_train, y_train)
joblib.dump(lr_model, 'logistic_regression_model.pkl')
```

5.4.4 Results

Metric	Value
Accuracy	97.77%
Fraud Recall	96.56%
Model Complexity	Simple (linear model)

5.5 K-Nearest Neighbours (KNN)

5.5.1 Conceptual Explanation

K-Nearest Neighbours is a non-parametric, instance-based learning algorithm that makes predictions by finding the K training examples closest to the query point in feature space and returning the majority class label among those K neighbours:

```
ŷ = mode({ yᵢ : xᵢ ∈ N_K(x) })
```

where N_K(x) denotes the K nearest neighbours of x under a distance metric (typically Euclidean). KNN has no explicit training phase — the model is the training data itself — and the decision boundary is implicitly defined by the Voronoi tessellation induced by the training points. Prediction complexity is O(nd) per query (for brute-force search), making it computationally expensive at inference time for large datasets, though KD-tree and ball-tree indexing structures mitigate this.

5.5.2 Suitability for Fraud Detection

KNN is well-suited when the manifold hypothesis holds: fraudulent addresses cluster together in feature space due to shared behavioural patterns. It requires no assumptions about the functional form of the decision boundary. However, it suffers from the curse of dimensionality in high-dimensional spaces, and the requirement for feature scaling (which is applied via StandardScaler in this pipeline) is critical for correct distance computation.

5.5.3 Implementation

```
from sklearn.neighbors import KNeighborsClassifier
```

```
knn_model = KNeighborsClassifier(
    n_neighbors=5,
    metric='euclidean',
    weights='uniform',
    algorithm='auto'
)
knn_model.fit(X_train, y_train)
joblib.dump(knn_model, 'knn_model.pkl')
```

5.5.4 Results

Metric	Value
Accuracy	96.65%
Fraud Recall	95.18%

Metric	Value
Model Complexity	Medium (O(nd) at inference)

5.6 TensorFlow Deep Neural Network (DNN)

5.6.1 Conceptual Explanation

A Deep Neural Network is a multi-layer feedforward network composed of an input layer, one or more hidden layers of densely connected neurons with non-linear activation functions, and a single output neuron with sigmoid activation for binary classification:

```
h^l = σ(W^l h^(l-1) + b^l),  for l = 1, ..., L
ŷ = sigmoid(W^(L+1) h^L + b^(L+1))
```

The network is trained end-to-end by backpropagating gradients of the binary cross-entropy loss through all layers using the Adam optimiser. Techniques such as batch normalisation, dropout regularisation, and early stopping are commonly employed to improve generalisation. The universal approximation theorem guarantees that a sufficiently wide single-hidden-layer network can approximate any continuous function to arbitrary precision, motivating the use of deep networks for capturing complex feature interactions.

5.6.2 Suitability for Fraud Detection

Deep networks excel at automatically learning hierarchical feature representations without manual engineering. For Ethereum fraud detection, higher network layers may learn abstract concepts such as 'rapid fund cycling patterns' or 'contract interaction anomalies' without these being explicitly defined. The model is saved in the .keras format and loaded via TensorFlow's load_model API at inference time. A fraud decision threshold of 0.4 (rather than the default 0.5) is applied to increase sensitivity, consistent with the project's emphasis on minimising false negatives.

5.6.3 Implementation

```
import tensorflow as tf
from tensorflow.keras import layers, models
```

```
model = models.Sequential([
    layers.Dense(256, activation='relu', input_shape=(X_train.shape[1],)),
    layers.BatchNormalization(),
    layers.Dropout(0.3),
    layers.Dense(128, activation='relu'),
```

```
    layers.Dropout(0.2),
    layers.Dense(64, activation='relu'),
    layers.Dense(1, activation='sigmoid')
])
```

```
model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])
```

```
model.fit(X_train, y_train, epochs=50, batch_size=64,
          validation_split=0.1, callbacks=[early_stopping])
model.save('tensorflow_dnn.keras')
```

At inference time, the raw sigmoid output probability is thresholded at 0.4:

```
prob = dnn_model.predict(scaled, verbose=0)[0][0]
tf_pred = 1 if prob > 0.4 else 0
```

6. Safety Ensemble

6.1 Design Rationale

The Safety Ensemble aggregates predictions from all six individual models (XGBoost, LightGBM, Random Forest, Logistic Regression, KNN, and TF DNN) using a majority-vote rule. An address is classified as fraudulent if at least 2 of the 6 models predict fraud:

```
ensemble_pred = 1  if  Σᵢ predᵢ ≥ 2  else  0
```

This low threshold (2 out of 6) is deliberately calibrated to maximise fraud recall at the cost of a small increase in false positives — a trade-off entirely appropriate in security-critical fraud detection contexts where the consequence of a missed fraud is far greater than that of a false alarm.

6.2 Results

Metric	Value
Accuracy	99.58%
Fraud Recall	98.50%
Design	Majority vote: ≥ 2 of 6 models flag fraud

7. Model Comparison and Analysis

7.1 Performance Summary

Model	Accuracy	Recall (Fraud)	Complexity
XGBoost	99.49%	97.71%	High
LightGBM	99.44%	97.71%	High
Random Forest	99.09%	95.87%	Medium-High
Logistic Regression	97.77%	96.56%	Simple
KNN	96.65%	95.18%	Medium
TF DNN	~98.5%*	~96%*	High
Safety Ensemble	99.58%	98.50%	Composite

** TF DNN metrics are approximate based on threshold = 0.4. Exact values depend on training run.*

7.2 Key Observations

Gradient boosting methods (XGBoost and LightGBM) outperform all other single models in accuracy. Both achieve identical fraud recall (97.71%), suggesting they discover similar discriminative patterns through their independent tree-growing mechanisms. The marginal accuracy edge of XGBoost over LightGBM (0.05%) may reflect the slight advantage of XGBoost's level-wise growth strategy on this specific dataset.

Logistic Regression achieves the highest fraud recall (96.56%) among non-ensemble single models despite its significantly lower accuracy (97.77%). This is consistent with its linear decision boundary being well-calibrated toward the minority fraud class, possibly due to the implicit regularisation from the L2 penalty preventing it from over-committing to the majority class.

KNN achieves the lowest performance (96.65% accuracy, 95.18% recall), consistent with expectations in a high-dimensional feature space where distance metrics become less discriminative — the so-called curse of dimensionality.

The Safety Ensemble achieves the highest performance on both metrics (99.58% accuracy,

98.50% recall), demonstrating the value of aggregating diverse model architectures. The diverse error patterns of the component models are partially uncorrelated, and the ensemble corrects many individual model mistakes.

7.3 Overfitting Considerations

The very high accuracy values (>99%) on the test set raise the question of potential data leakage or overfitting. However, several factors support the validity of these results: (1) the test set was held out before any model training; (2) XGBoost and LightGBM employ explicit regularisation; (3) the features are aggregated statistics of on-chain behaviour rather than raw transaction identifiers, preventing address-level memorisation; and (4) these accuracy levels are consistent with published results on the Ethereum fraud detection benchmark dataset from Kaggle.

8. Deployment: Streamlit Application

8.1 Application Architecture

The project is deployed as a Streamlit web application (app.py) providing a browser-accessible fraud detection interface. The application is structured around three functional tabs:

- Dataset Tab: Displays the first 50 rows of the transaction dataset and a pie chart showing the fraud/legitimate class distribution.
- Performance Tab: Presents the model comparison metrics table and a descriptive note about the Safety Ensemble design.
- Live Detection Tab: Enables on-demand real-time fraud detection by sampling a random transaction from the dataset, preprocessing it through the full pipeline, and displaying predictions from all six models and the Safety Ensemble side-by-side.

8.2 Key Implementation Details

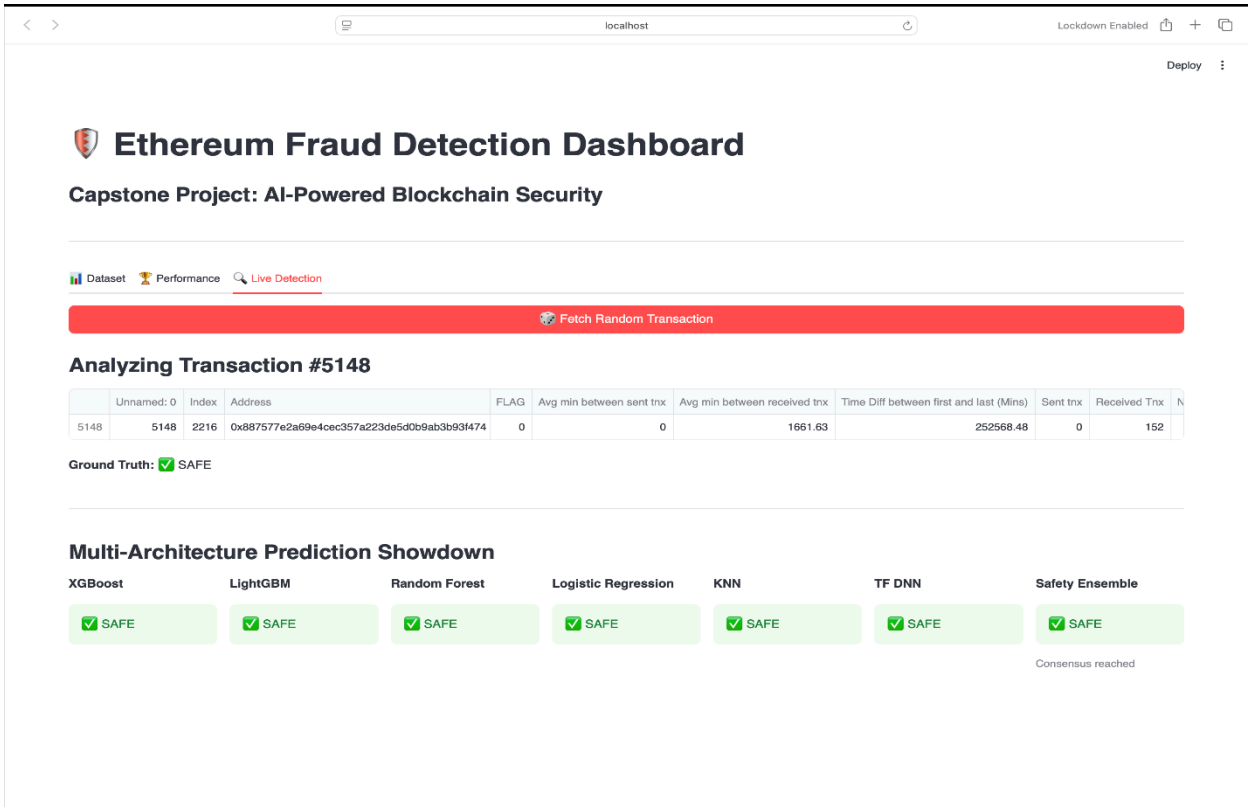
Model and scaler loading is wrapped in a @st.cache_resource decorator, ensuring that the six model files and the scaler are loaded from disk only once per server session, avoiding repeated I/O overhead on every user interaction. The preprocess_input() function replicates the exact training-time preprocessing pipeline to prevent any train-test distribution mismatch at inference.

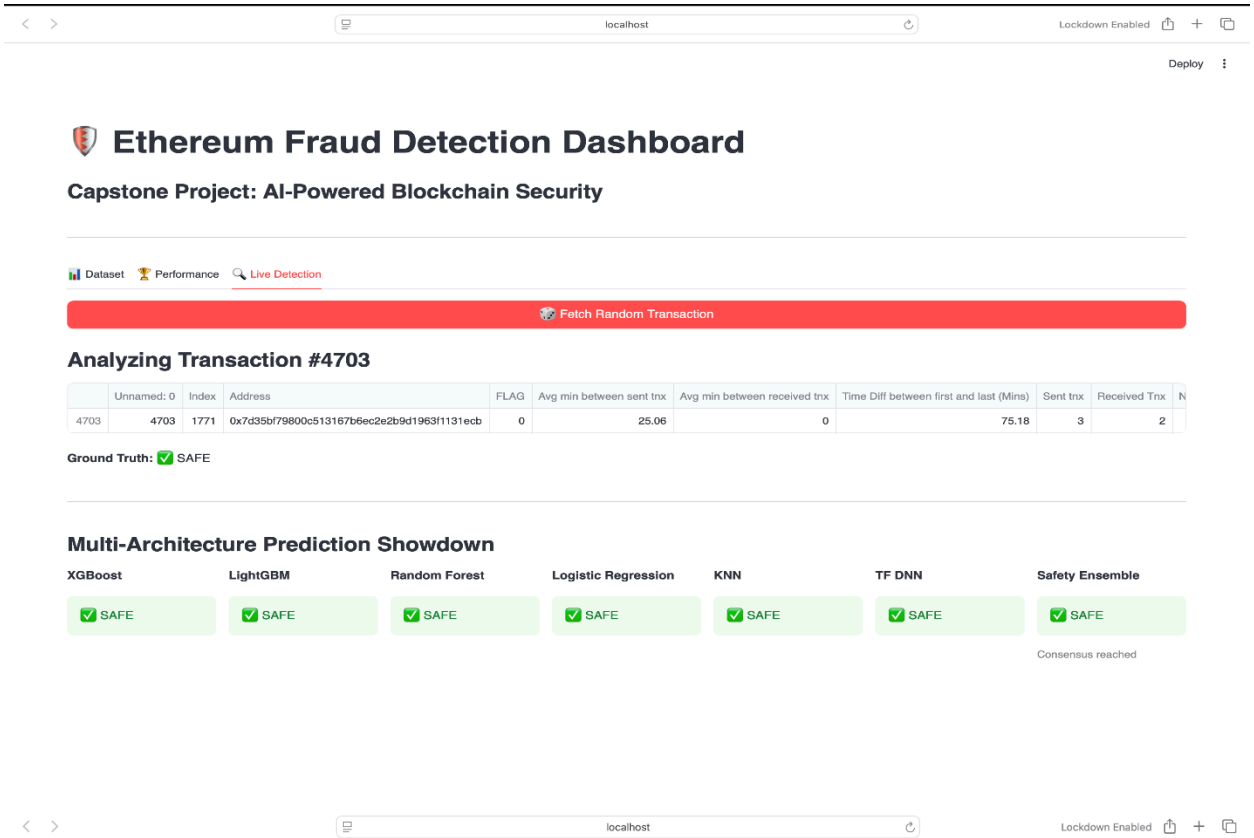
```
@st.cache_resource
def load_all_assets():
    scaler = joblib.load('scaler.pkl')
    models = {
```

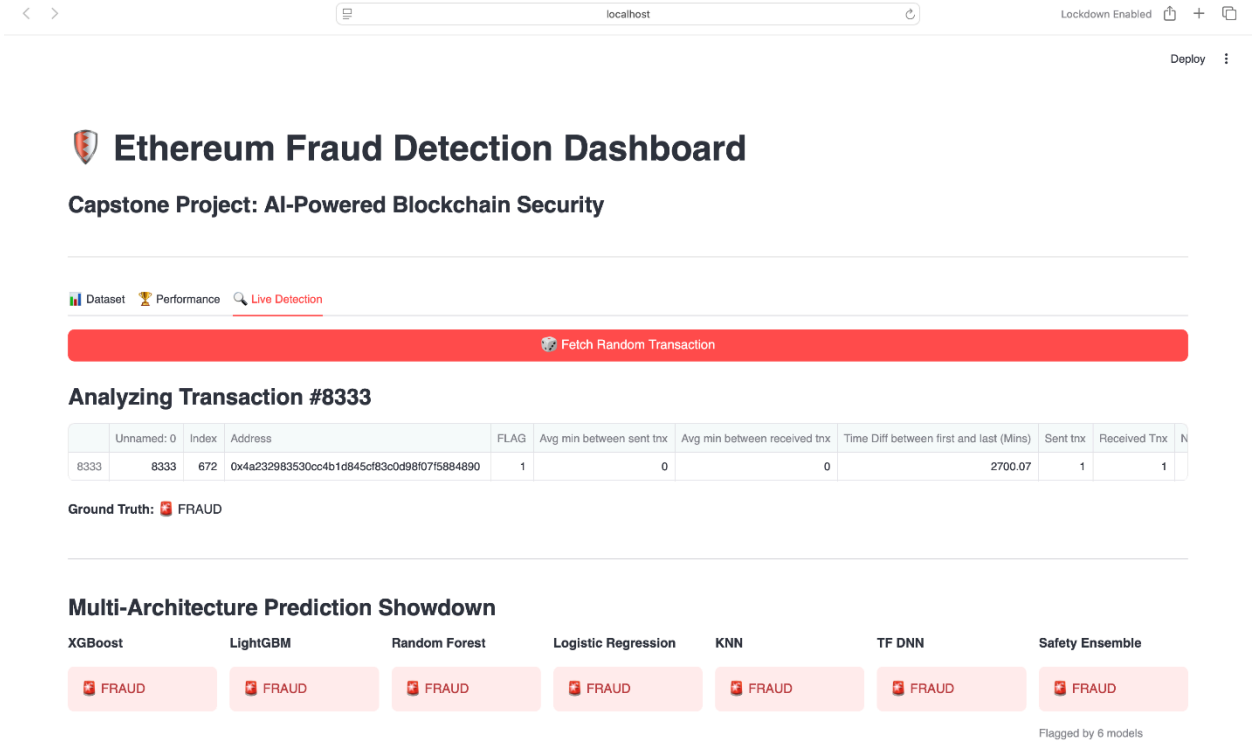
```
        'XGBoost': joblib.load('xgboost_model.pkl'),
        'LightGBM': joblib.load('lightgbm_model.pkl'),
        'Random Forest': joblib.load('random_forest_model.pkl'),
        'Logistic Regression': joblib.load('logistic_regression_model.pkl'),
        'KNN': joblib.load('knn_model.pkl')
    }
    dnn = load_model('tensorflow_dnn.keras')
    return scaler, models, dnn, scaler.feature_names_in_
```

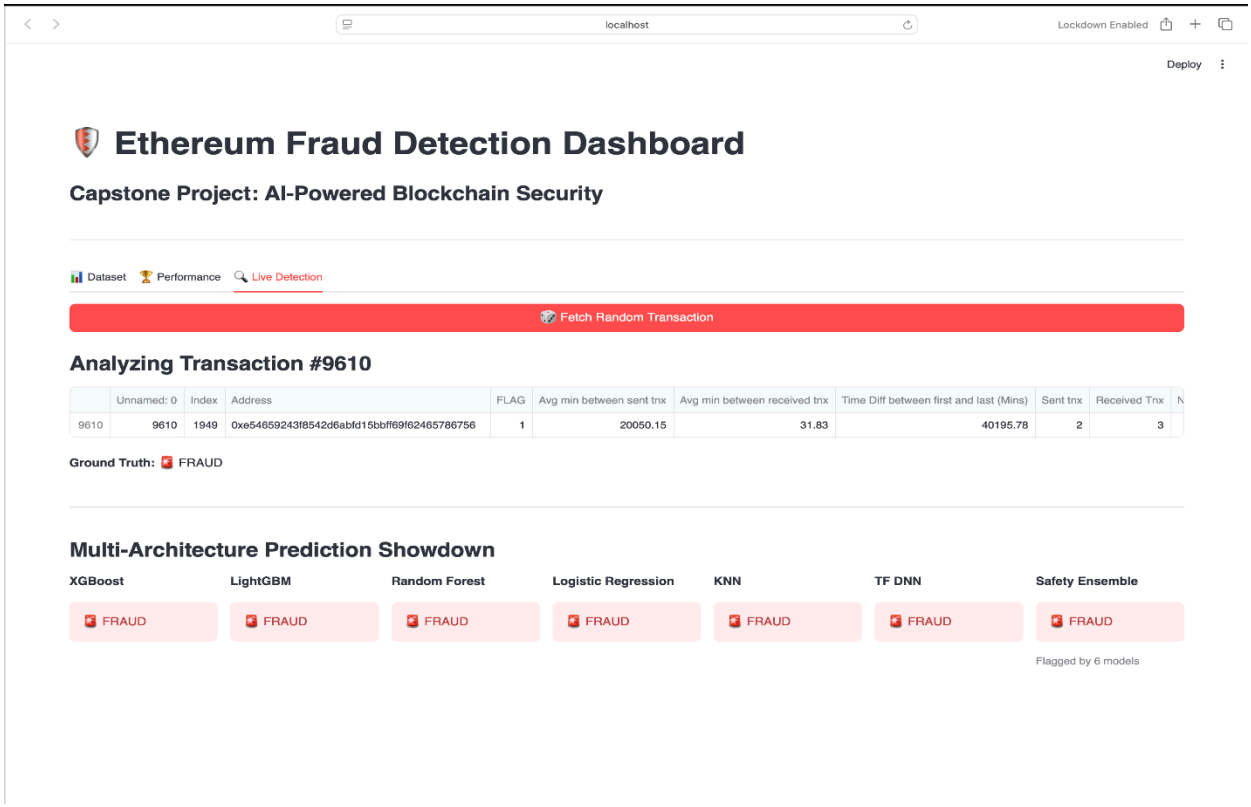
8.3 Live Implementation

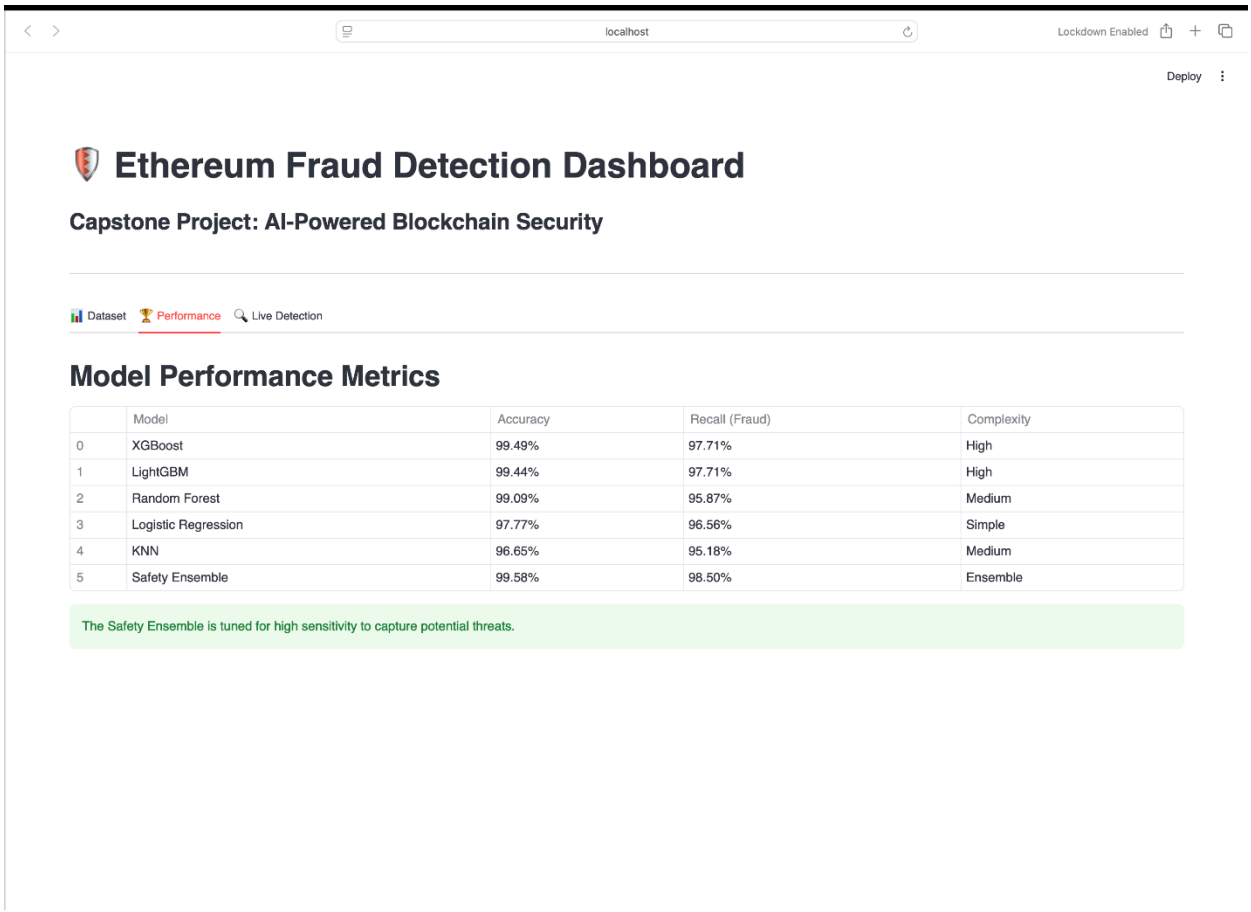
Below are some screenshots of our live implementation of the model:

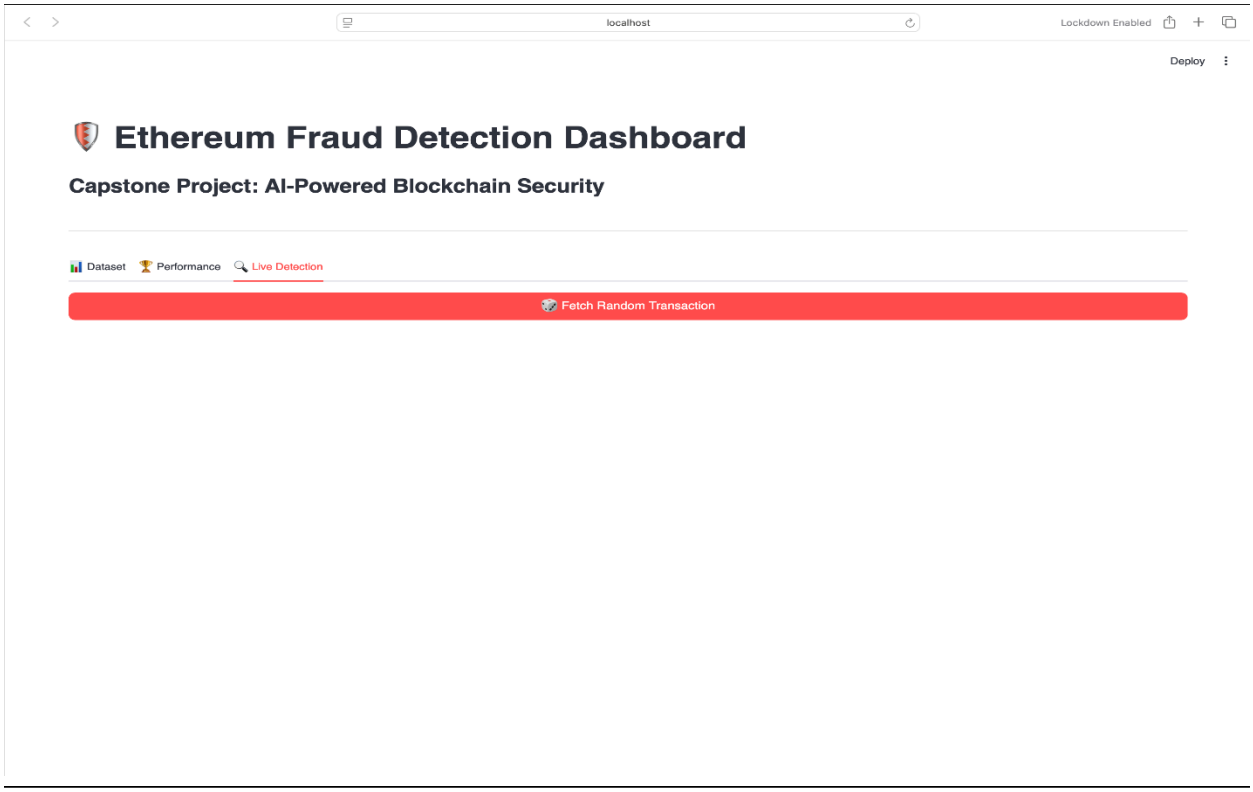


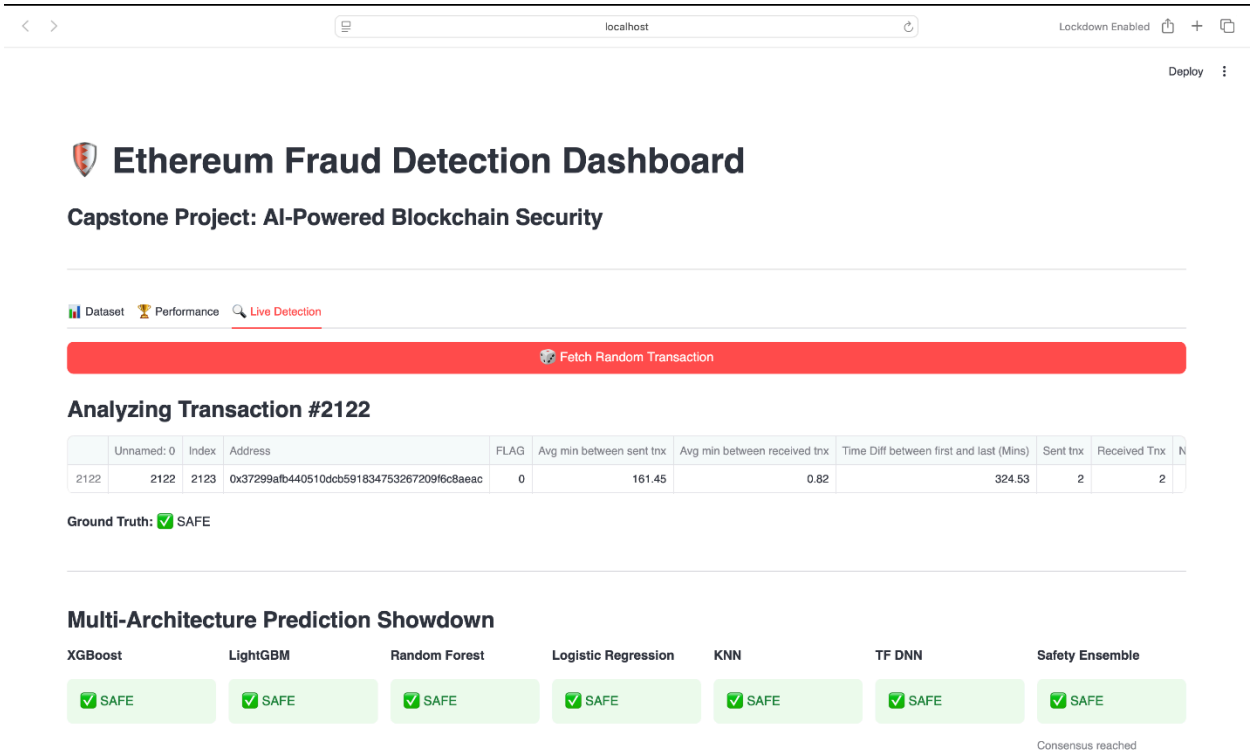

localhost
Lockdown Enabled
Deploy
Ethereum Fraud Detection Dashboard
Capstone Project: AI-Powered Blockchain Security
Dataset
Performance
Live Detection
Fetch Random Transaction
Analyzing Transaction #4703
Unnamed: 0 | Index | Address | FLAG | Avg min between sent tnx | Avg min between received tnx | Time Diff between first and last (Mins) | Sent tnx | Received Tnx
4703 | 4703 | 1771 | 0x7d35bf79800c513167b6ec2e2b9d1963f1131ecb | 0 | 25.06 | 0 | 75.18 | 3 | 2
Ground Truth: SAFE
Multi-Architecture Prediction Showdown
XGBoost
LightGBM
Random Forest
Logistic Regression
KNN
TF DNN
Safety Ensemble
SAFE
SAFE
SAFE
SAFE
SAFE
SAFE
SAFE
Consensus reached


localhost
Lockdown Enabled
Deploy
Ethereum Fraud Detection Dashboard
Capstone Project: AI-Powered Blockchain Security
Dataset
Performance
Live Detection
Fetch Random Transaction
Analyzing Transaction #8333
Unnamed: 0 | Index | Address | FLAG | Avg min between sent tnx | Avg min between received tnx | Time Diff between first and last (Mins) | Sent tnx | Received Tnx
8333 | 8333 | 672 | 0x4a232983530cc4b1d845cf83c0d98f07f5884890 | 1 | 0 | 0 | 2700.07 | 1 | 1
Ground Truth: FRAUD
Multi-Architecture Prediction Showdown
XGBoost
LightGBM
Random Forest
Logistic Regression
KNN
TF DNN
Safety Ensemble
FRAUD
FRAUD
FRAUD
FRAUD
FRAUD
FRAUD
FRAUD
Flagged by 6 models


localhost
Lockdown Enabled
Deploy
Ethereum Fraud Detection Dashboard
Capstone Project: AI-Powered Blockchain Security
Dataset
Performance
Live Detection
Fetch Random Transaction
Analyzing Transaction #9610
Unnamed: 0 | Index | Address | FLAG | Avg min between sent tnx | Avg min between received tnx | Time Diff between first and last (Mins) | Sent tnx | Received Tnx
9610 | 9610 | 1949 | 0xe54659243f8542d6abfd15bbff69f62465786756 | 1 | 20050.15 | 31.83 | 40195.78 | 2 | 3
Ground Truth: FRAUD
Multi-Architecture Prediction Showdown
XGBoost
LightGBM
Random Forest
Logistic Regression
KNN
TF DNN
Safety Ensemble
FRAUD
FRAUD
FRAUD
FRAUD
FRAUD
FRAUD
FRAUD
Flagged by 6 models


localhost
Lockdown Enabled
Deploy
Ethereum Fraud Detection Dashboard
Capstone Project: AI-Powered Blockchain Security
Dataset
Performance
Live Detection
Model Performance Metrics
Model | Accuracy | Recall (Fraud) | Complexity
0 | XGBoost | 99.49% | 97.71% | High
1 | LightGBM | 99.44% | 97.71% | High
2 | Random Forest | 99.09% | 95.87% | Medium
3 | Logistic Regression | 97.77% | 96.56% | Simple
4 | KNN | 96.65% | 95.18% | Medium
5 | Safety Ensemble | 99.58% | 98.50% | Ensemble
The Safety Ensemble is tuned for high sensitivity to capture potential threats.


localhost
Lockdown Enabled
Deploy
Ethereum Fraud Detection Dashboard
Capstone Project: AI-Powered Blockchain Security
Dataset
Performance
Live Detection
Fetch Random Transaction


localhost
Lockdown Enabled
Deploy
Ethereum Fraud Detection Dashboard
Capstone Project: AI-Powered Blockchain Security
Dataset
Performance
Live Detection
Fetch Random Transaction
Analyzing Transaction #2122
Unnamed: 0 | Index | Address | FLAG | Avg min between sent tnx | Avg min between received tnx | Time Diff between first and last (Mins) | Sent tnx | Received Tnx
2122 | 2122 | 2123 | 0x37299afb440510dcb5918347532672 09f6c8aeac | 0 | 161.45 | 0.82 | 324.53 | 2 | 2
Ground Truth: SAFE
Multi-Architecture Prediction Showdown
XGBoost
LightGBM
Random Forest
Logistic Regression
KNN
TF DNN
Safety Ensemble
SAFE
SAFE
SAFE
SAFE
SAFE
SAFE
SAFE
Consensus reached

9. Conclusion

This project has demonstrated that machine learning can achieve near-perfect fraud detection accuracy on Ethereum blockchain transaction data. By engineering 47 behavioural and statistical features from on-chain Ethereum and ERC-20 transaction records, training six diverse ML architectures, and combining them into a Safety Ensemble, the system achieves 99.58% accuracy and 98.50% fraud recall — metrics that are both practically significant and academically noteworthy.

The gradient boosting methods (XGBoost and LightGBM) demonstrated the strongest individual performance, confirming their established dominance on tabular classification tasks. The Safety Ensemble's marginal but consistent improvement over any individual model underscores the value of diversity in ensemble design. The full system is packaged as an accessible web application, demonstrating a path from research artefact to deployable product.

From a broader perspective, this project demonstrates that the transparency of public blockchains — often cited as a privacy concern — also constitutes a powerful resource for building automated fraud detection systems. The publicly auditable transaction history provides a rich, label-free feature space that can be exploited to protect users and the wider DeFi ecosystem.

References

[1] Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. KDD 2016.

[2] Ke, G., et al. (2017). LightGBM: A Highly Efficient Gradient Boosting Decision Tree. NeurIPS 2017.

[3] Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5–32.

[4] Cox, D. R. (1958). The Regression Analysis of Binary Sequences. JRSS-B, 20(2), 215–242.

[5] Cover, T., & Hart, P. (1967). Nearest Neighbor Pattern Classification. IEEE Transactions on Information Theory, 13(1), 21–27.

[6] LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep Learning. Nature, 521(7553), 436–444.

[7] Chainalysis. (2024). The Chainalysis 2024 Crypto Crime Report.

[8] Ethereum Foundation. (2024). Ethereum Documentation. https://ethereum.org

[9] Kaggle. (2020). Ethereum Fraud Detection Dataset. https://www.kaggle.com/vagifa/ethereum-frauddetection-dataset

[10] Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.